```
    return NULL; //[cite: 1]
}
```

Important Things to Keep in Mind When Using This Project

1. **Path Syntax Differences:**

   If you use `<stdio.h>` functions (`fopen` , `remove`), **always** prefix your paths with `"fat:0:"` (e.g., `"fat:0:data.bin"`) so the TI runtime directs it to `ffcio.c` [cite: 1, 7].

   If you use native FatFs functions (`f_open` , `f_unlink`), **only** use `"0:"` (e.g., `"0:data.bin"`). fatsd.c + 2

2. **Sector Size vs. Erase Block Size:** diskio.c

   The filesystem acts like it has **512-byte sectors**, but the MSP432E4 hardware can only erase in **16 KB blocks.** diskio.c

   Every time you modify or append to a file, `disk_write()` copies 16 KB of flash into RAM (`s_blockBuffer`), modifies the targeted 512-byte chunk, erases the 16 KB flash block, and writes it all back. diskio.c

   Best Practice: Group your writes into bigger chunks and avoid constantly opening, appending small 1-byte entries, and closing files in tight loops to prevent premature flash wear and sluggish performance. diskio.c

3. **Storage Boundary:**

   The disk is capped at **256 KB** (512 sectors × 512 bytes) starting at memory address `0x000C0000` . Do not place application firmware code or constants in linker script regions overlapping `0x000C0000` – `0x00100000` as `f_mkfs` and writes

   take referrence of uploaded source files i want two seperate projects on one create a file and write some data and another one will open that file read data from it . executing one project and turn off and another project what changes had to be made

To split your codebase into two independent programs—**Project 1 (Writer)** and **Project 2 (Reader)**—where data persists in the MCU's internal flash across power cycles, you only

need to modify `fatsd.c` for each project and configure their behavior properly. fatsd.c + 1

All other files (`main_freertos.c` , `diskio.c` , `diskio.h` , `ff.c` , `ff.h` , `ffcio.c` , `ffcio.h` , `ffconf.h` , `ffunicode.c`) remain **identical** across both projects. main_freertos.c + 4

Critical Changes You Must Make Between Projects

1. **Remove `f_mkfs` from Project 2 (Reader):**

   In `fatsd.c` , the original code formats the partition if it does not find an active filesystem. fatsd.c

   Project 1 (Writer) formats the drive the first time it boots. fatsd.c

   Project 2 (Reader) MUST NOT call `f_mkfs()` . If the reader formats the flash, it will wipe out the file created by Project 1. It should simply report an error and stop if no filesystem exists. fatsd.c + 3

2. **Prevent Flasher/Debugger from Erasing the Filesystem Region:**

   The filesystem lives at the last 256 KB of MCU flash: `0x000C0000` to `0x00100000` . diskio.c

   When loading Project 2 onto the board, your IDE/programmer (CCS, UniFlash, or J-Link) must **not perform a complete chip erase**.

   In Code Composer Studio (CCS):

   Go to **Project Properties** → **Debug** → **Flash Settings** (under the target configuration).

   Set the Erase settings to **"Erase necessary sectors only"** (do NOT choose *"Entire Flash"*). This ensures flashing Project 2 does not clear the flash memory space occupied by the FAT drive. diskio.c

Project 1: The File Writer (`fatsd.c`)

This project formats the flash (if blank), creates a file named `data.txt` , writes persistent information to it, flushes the data to physical flash, and unmounts the volume. fatsd.c + 4

Replace the contents of `fatsd.c` in your first project with the following:

C

```c
#include <file.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#include <FreeRTOS.h>
#include <semphr.h>
#include <fat_driver/ff.h>
#include <fat_driver/ffcio.h>
#include <ti/display/Display.h>
#include <ti/drivers/GPIO.h>
#include "ti_drivers_config.h"

#define DRIVE_NUM       0
#define TARGET_FILE     "fat:0:data.txt"

static Display_Handle display;
static FATFS g_fatFs;

const struct timespec ts = {
    .tv_sec = 1762700000,
    .tv_nsec = 0
};

char fatfsPrefix[] = "fat";

/* Data payload to be preserved across reboot */
const char payloadData[] =
"================================================\n"
"PROJECT 1: Persistent Data Storage Test\n"
"Board: MSP432E401Y Internal Flash (256 KB)\n"
"Status: Written successfully and power preserved!\n"
"================================================\n";

void *mainThread(void *arg0)
{
    FILE *fp;
    FRESULT fresult;
```

```
    GPIO_init();
    Display_init();

    GPIO_setConfig(CONFIG_GPIO_LED_0, GPIO_CFG_OUT_STD | GPIO_CFG_OUT_LOW);

    /* Bind TI RTS C-IO functions to FatFs */
    add_device(fatfsPrefix, _MSA, ffcio_open, ffcio_close, ffcio_read,
               ffcio_write, ffcio_lseek, ffcio_unlink, ffcio_rename);

    display = Display_open(Display_Type_UART, NULL);
    if (display == NULL) {
        while (1);
    }

    clock_settime(CLOCK_REALTIME, &ts);
    GPIO_write(CONFIG_GPIO_LED_0, CONFIG_GPIO_LED_ON);

    Display_printf(display, 0, 0, "\n=== PROJECT 1: FILE WRITER ===\n");

    /* 1. Mount the internal flash drive */
    fresult = f_mount(&g_fatFs, "0:", 1);
    if (fresult == FR_NO_FILESYSTEM) {
        Display_printf(display, 0, 0, "No filesystem found. Formatting 256KB flas
        BYTE workBuf[FF_MAX_SS];
        fresult = f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf));
        if (fresult != FR_OK) {
            Display_printf(display, 0, 0, "f_mkfs failed: %d\n", fresult);
            while (1);
        }
        Display_printf(display, 0, 0, "Format complete. Re-mounting...\n");
        fresult = f_mount(&g_fatFs, "0:", 1);
    }

    if (fresult != FR_OK) {
        Display_printf(display, 0, 0, "Mount failed: %d\n", fresult);
        while (1);
    }
    Display_printf(display, 0, 0, "Flash drive 0 mounted.\n");

    /* 2. Create and write to the file */
    Display_printf(display, 0, 0, "Opening \"%s\" for write...\n", TARGET_FILE);
    fp = fopen(TARGET_FILE, "w");
    if (!fp) {
        Display_printf(display, 0, 0, "Error: Could not open \"%s\" for write!\n"
        while (1);
    }
```

```
    size_t written = fwrite(payloadData, 1, strlen(payloadData), fp);
    Display_printf(display, 0, 0, "Written %u bytes.\n", (unsigned int)written);

    /* Flush and close so data is committed to on-chip flash */
    fflush(fp);
    fclose(fp);
    Display_printf(display, 0, 0, "File written and closed successfully.\n");

    /* 3. Unmount cleanly */
    f_mount(NULL, "0:", 0);
    Display_printf(display, 0, 0, "Drive unmounted.\n");
    Display_printf(display, 0, 0, "You can now POWER OFF the board.\n");

    return NULL;
}

/* Time provider required by FatFs */
int32_t fatfs_getFatTime(void)
{
    time_t seconds = time(NULL);
    struct tm *pTime = localtime(&seconds);

    uint32_t fatTime = ((uint32_t)(pTime->tm_year - 80) << 25) |
                       ((uint32_t)(pTime->tm_mon) << 21) |
                       ((uint32_t)(pTime->tm_mday) << 16) |
                       ((uint32_t)(pTime->tm_hour) << 11) |
                       ((uint32_t)(pTime->tm_min) << 5) |
                       ((uint32_t)(pTime->tm_sec) >> 1);

    return ((int32_t)fatTime);
}

DWORD get_fattime(void)
{
    return (DWORD)fatfs_getFatTime();
}

#if FF_FS_REENTRANT
int ff_cre_syncobj(BYTE vol, FF_SYNC_t *sobj)
{
    *sobj = xSemaphoreCreateMutex();
    return (*sobj != NULL) ? 1 : 0;
}

int ff_del_syncobj(FF_SYNC_t sobj)
```

```
{
    vSemaphoreDelete(sobj);
    return 1;
}

int ff_req_grant(FF_SYNC_t sobj)
{
    return (xSemaphoreTake(sobj, pdMS_TO_TICKS(FF_FS_TIMEOUT)) == pdTRUE) ? 1 : 0
}

void ff_rel_grant(FF_SYNC_t sobj)
{
    xSemaphoreGive(sobj);
}
#endif
```

Project 2: The File Reader (`fatsd.c`)

This project mounts the drive, checks for the existing `data.txt` file created by Project 1, reads its contents into RAM, prints the data to the UART terminal, and unmounts. Notice that `f_mkfs` **has been removed.** fatsd.c + 4

Replace the contents of `fatsd.c` in your second project with the following:

c

```c
#include <file.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#include <FreeRTOS.h>
#include <semphr.h>
#include <fat_driver/ff.h>
#include <fat_driver/ffcio.h>
#include <ti/display/Display.h>
#include <ti/drivers/GPIO.h>
```

```c
#include "ti_drivers_config.h"

#define TARGET_FILE     "fat:0:data.txt"
#define READ_BUFF_SIZE  512

static Display_Handle display;
static FATFS g_fatFs;
static char read_buffer[READ_BUFF_SIZE];

char fatfsPrefix[] = "fat";

void *mainThread(void *arg0)
{
    FILE *fp;
    FRESULT fresult;
    size_t bytesRead;

    GPIO_init();
    Display_init();

    GPIO_setConfig(CONFIG_GPIO_LED_0, GPIO_CFG_OUT_STD | GPIO_CFG_OUT_LOW);

    /* Bind TI RTS C-IO functions to FatFs */
    add_device(fatfsPrefix, _MSA, ffcio_open, ffcio_close, ffcio_read,
               ffcio_write, ffcio_lseek, ffcio_unlink, ffcio_rename);

    display = Display_open(Display_Type_UART, NULL);
    if (display == NULL) {
        while (1);
    }

    GPIO_write(CONFIG_GPIO_LED_0, CONFIG_GPIO_LED_ON);

    Display_printf(display, 0, 0, "\n=== PROJECT 2: FILE READER ===\n");

    /* 1. Mount the existing flash drive (DO NOT FORMAT) */
    fresult = f_mount(&g_fatFs, "0:", 1);
    if (fresult == FR_NO_FILESYSTEM) {
        Display_printf(display, 0, 0, "Error: No filesystem found! (Did you run P
        while (1);
    } else if (fresult != FR_OK) {
        Display_printf(display, 0, 0, "Mount failed with code: %d\n", fresult);
        while (1);
    }
    Display_printf(display, 0, 0, "Flash drive 0 mounted successfully.\n");
```

```
    /* 2. Open the file written by Project 1 */
    Display_printf(display, 0, 0, "Opening \"%s\" for reading...\n", TARGET_FILE)
    fp = fopen(TARGET_FILE, "r");
    if (!fp) {
        Display_printf(display, 0, 0, "Error: \"%s\" not found on disk!\n", TARGE
        f_mount(NULL, "0:", 0);
        while (1);
    }

    /* 3. Stream data to UART */
    Display_printf(display, 0, 0, "\n--- File Contents Start ---\n");
    while ((bytesRead = fread(read_buffer, 1, READ_BUFF_SIZE - 1, fp)) > 0) {
        read_buffer[bytesRead] = '\0';
        Display_printf(display, 0, 0, "%s", read_buffer);
    }
    Display_printf(display, 0, 0, "--- File Contents End ---\n\n");

    fclose(fp);

    /* 4. Unmount cleanly */
    f_mount(NULL, "0:", 0);
    Display_printf(display, 0, 0, "Drive unmounted successfully.\n");

    return NULL;
}

int32_t fatfs_getFatTime(void)
{
    return 0;
}

DWORD get_fattime(void)
{
    return 0;
}

#if FF_FS_REENTRANT
int ff_cre_syncobj(BYTE vol, FF_SYNC_t *sobj)
{
    *sobj = xSemaphoreCreateMutex();
    return (*sobj != NULL) ? 1 : 0;
}

int ff_del_syncobj(FF_SYNC_t sobj)
{
    vSemaphoreDelete(sobj);
```

```
    return 1;
}

int ff_req_grant(FF_SYNC_t sobj)
{
    return (xSemaphoreTake(sobj, pdMS_TO_TICKS(FF_FS_TIMEOUT)) == pdTRUE) ? 1 : 0
}

void ff_rel_grant(FF_SYNC_t sobj)
{
    xSemaphoreGive(sobj);
}
#endif
```

Step-by-Step Execution Workflow

1. **Flash Project 1 (Writer)** onto the board using CCS. main_freertos.c
2. Open your UART serial terminal (115200 baud). You will see: fatsd.c

Plaintext

```
=== PROJECT 1: FILE WRITER ===
No filesystem found. Formatting 256KB flash area...
Format complete. Re-mounting...
Flash drive 0 mounted.
Opening "fat:0:data.txt" for write...
Written 193 bytes.
File written and closed successfully.
Drive unmounted.
You can now POWER OFF the board.
```

3. **Power down / Disconnect** the LaunchPad USB cable completely. The data is now retained in on-chip flash at `0x000C0000`. diskio.c
4. Reconnect the board to your PC.
5. In CCS for **Project 2**, verify **"Erase Necessary Sectors Only"** is selected in your Debugger Flash Settings so the IDE does not wipe out address `0x000C0000`. diskio.c
6. **Flash and Run Project 2 (Reader).** main_freertos.c

7. The terminal will output:

Plaintext

```
=== PROJECT 2: FILE READER ===
Flash drive 0 mounted successfully.
Opening "fat:0:data.txt" for reading...

--- File Contents Start ---
================================================
PROJECT 1: Persistent Data Storage Test
Board: MSP432E401Y Internal Flash (256 KB)
Status: Written successfully and power preserved!
================================================
--- File Contents End ---

Drive unmounted successfully.
```